Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error
```

```
# 1. Upload dataset
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

```
Choose Files uber.csv
uber.csv(text/csv) - 23458139 bytes, last modified: 21/10/2026 - 100% done
Saving uber.csv to uber (1).csv
Dataset Shape: (200000, 9)
   Unnamed: 0                            key  fare_amount  \
0    24238194    2015-05-07 19:52:06.0000003          7.5
1    27835199    2009-07-17 20:04:56.0000002          7.7
2    44984355   2009-08-24 21:45:00.00000061         12.9
3    25894730    2009-06-26 08:22:21.0000001          5.3
4    17610152  2014-08-28 17:47:00.000000188         16.0

           pickup_datetime  pickup_longitude  pickup_latitude  \
0  2015-05-07 19:52:06 UTC        -73.999817        40.738354
1  2009-07-17 20:04:56 UTC        -73.994355        40.728225
2  2009-08-24 21:45:00 UTC        -74.005043        40.740770
3  2009-06-26 08:22:21 UTC        -73.976124        40.790844
4  2014-08-28 17:47:00 UTC        -73.925023        40.744085

   dropoff_longitude  dropoff_latitude  passenger_count
0         -73.999512         40.723217                1
1         -73.994710         40.750325                1
2         -73.962565         40.772647                1
3         -73.965316         40.803349                3
4         -73.973082         40.761247                5
```

```
# 2. Preprocess dataset
df = df.dropna()

df = df[
    (df["fare_amount"] > 0) &
    (df["fare_amount"] <= 500) &
    (df["passenger_count"] > 0)
]

# Remove invalid coordinates
df = df[
    df["pickup_longitude"].between(-180, 180) &
    df["dropoff_longitude"].between(-180, 180) &
    df["pickup_latitude"].between(-90, 90) &
    df["dropoff_latitude"].between(-90, 90)
]

print("Shape after cleaning:", df.shape)
```

```
Shape after cleaning: (199257, 9)
```

```
# 3. Identify outliers using IQR
Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)
IQR = Q3 - Q1

df = df[
    df["fare_amount"].between(
        Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    )
]

print("Shape after outlier removal:", df.shape)
```

```
Shape after outlier removal: (182137, 9)
```

```
# 4. Check correlation
features = [
    "pickup_longitude", "pickup_latitude",
    "dropoff_longitude", "dropoff_latitude",
    "passenger_count"
]

print("\nCorrelation Matrix:")
print(df[features + ["fare_amount"]].corr().round(2))
```

```
Correlation Matrix:
                   pickup_longitude  pickup_latitude  dropoff_longitude  \
pickup_longitude               1.00            -0.98               0.96
pickup_latitude               -0.98             1.00              -0.94
dropoff_longitude              0.96            -0.94               1.00
dropoff_latitude              -0.94             0.96              -0.98
passenger_count               -0.00            -0.00              -0.00
fare_amount                    0.00            -0.00              -0.00

                   dropoff_latitude  passenger_count  fare_amount
pickup_longitude              -0.94            -0.00         0.00
pickup_latitude                0.96            -0.00        -0.00
dropoff_longitude             -0.98            -0.00        -0.00
dropoff_latitude               1.00            -0.00         0.00
passenger_count               -0.00             1.00         0.01
fare_amount                    0.00             0.01         1.00
```

```
# 5. Split dataset
X = df[features]
y = df["fare_amount"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 6. Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)
```

```
# 7. Random Forest Regression
rf = RandomForestRegressor(
    n_estimators=50, random_state=42
)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
```

```
# 8. Evaluate models
print("\nLinear Regression")
print("R2 Score:", round(r2_score(y_test, pred_lr), 3))
print("RMSE:", round(np.sqrt(
    mean_squared_error(y_test, pred_lr)
), 3))

print("\nRandom Forest Regression")
print("R2 Score:", round(r2_score(y_test, pred_rf), 3))
print("RMSE:", round(np.sqrt(
    mean_squared_error(y_test, pred_rf)
), 3))
```

```
Linear Regression
R2 Score: 0.0
RMSE: 4.137

Random Forest Regression
R2 Score: 0.706
RMSE: 2.245
```

Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 2

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

```
Choose Files emails.csv
emails.csv(text/csv) - 31243156 bytes, last modified: 21/10/2026 - 100% done
Saving emails.csv to emails.csv
Dataset Shape: (5172, 3002)
  Email No.  the  to  ect  and  for  of    a  you  hou  ...  connevey  jay  \
0   Email 1    0   0    1    0    0   0    2    0    0  ...         0    0
1   Email 2    8  13   24    6    6   2  102    1   27  ...         0    0
2   Email 3    0   0    1    0    0   0    8    0    0  ...         0    0
3   Email 4    0   5   22    0    5   1   51    2   10  ...         0    0
4   Email 5    7   6   17    1    5   2   57    0    9  ...         0    0

   valued  lay  infrastructure  military  allowing  ff  dry  Prediction
0       0    0               0         0         0   0    0           0
1       0    0               0         0         0   1    0           0
2       0    0               0         0         0   0    0           0
3       0    0               0         0         0   0    0           0
4       0    0               0         0         0   1    0           0

[5 rows x 3002 columns]
```

```
X = df.iloc[:, 1:-1]
y = df.iloc[:, -1]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
print("\nClass Distribution:")
print(y.value_counts())
```

```
Features Shape: (5172, 3000)
Target Shape: (5172,)

Class Distribution:
Prediction
0    3672
1    1500
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (4137, 3000)
Testing Data: (1035, 3000)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Feature scaling completed.")
```

```
Feature scaling completed.
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("KNN Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred))
```

```
KNN Accuracy: 0.8338164251207729
KNN Confusion Matrix:
[[576 159]
 [ 13 287]]
```

```
svm = SVC(kernel="linear")
svm.fit(X_train, y_train)

svm_pred = svm.predict(X_test)

print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
print("SVM Confusion Matrix:")
print(confusion_matrix(y_test, svm_pred))
```

```
SVM Accuracy: 0.9507246376811594
SVM Confusion Matrix:
[[709  26]
 [ 25 275]]
```

```
print("Model Performance Comparison")
print("----------------------------")
print("KNN Accuracy:", round(accuracy_score(y_test, knn_pred), 4))
print("SVM Accuracy:", round(accuracy_score(y_test, svm_pred), 4))
```

```
Model Performance Comparison
----------------------------
KNN Accuracy: 0.8338
SVM Accuracy: 0.9507
```

Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 3

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

Choose Files Churn_Modelling.csv

Churn_Modelling.csv(text/csv) - 684858 bytes, last modified: 21/10/2026 - 100% done

```
Saving Churn_Modelling.csv to Churn_Modelling.csv
Dataset Shape: (10000, 14)
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited
0        101348.88       1
1        112542.58       0
2        113931.57       1
3         93826.63       0
4         79084.10       0
```

```
df = df.dropna()

X = df.drop(
    columns=["Exited", "RowNumber", "CustomerId", "Surname"],
    errors="ignore"
)
y = df["Exited"]

X = pd.get_dummies(X, drop_first=True, dtype=int)
```

```
print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
print("\nClass Distribution:")
print(y.value_counts())
```

```
Features Shape: (10000, 11)
Target Shape: (10000,)

Class Distribution:
Exited
0    7963
1    2037
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (8000, 11)
Testing Data: (2000, 11)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Data normalization completed.")
```

```
Data normalization completed.
```

```
model = Sequential([
    Dense(16, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	192
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 1)	9

Total params: 337 (1.32 KB)
Trainable params: 337 (1.32 KB)
Non-trainable params: 0 (0.00 B)

```
history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.7944 - loss: 0.5165 -
Epoch 2/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.8077 - loss: 0.4439 -
Epoch 3/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8192 - loss: 0.4247 -
Epoch 4/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.8231 - loss: 0.4142 -
Epoch 5/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8292 - loss: 0.4054 -
Epoch 6/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8341 - loss: 0.3965 -
Epoch 7/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8386 - loss: 0.3876 -
Epoch 8/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8402 - loss: 0.3794 -
Epoch 9/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8462 - loss: 0.3712 -
Epoch 10/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8481 - loss: 0.3646 -
Epoch 11/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8505 - loss: 0.3599 -
Epoch 12/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8506 - loss: 0.3554 -
Epoch 13/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8508 - loss: 0.3523 -
Epoch 14/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8519 - loss: 0.3495 -
Epoch 15/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8534 - loss: 0.3480 -
Epoch 16/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8567 - loss: 0.3458 -
Epoch 17/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8561 - loss: 0.3447 -
```

```
Epoch 18/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8548 - loss: 0.3435 -
Epoch 19/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8573 - loss: 0.3419 -
Epoch 20/20
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8578 - loss: 0.3407 -
```

```
y_prob = model.predict(X_test)
y_pred = (y_prob > 0.5).astype(int).ravel()

print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

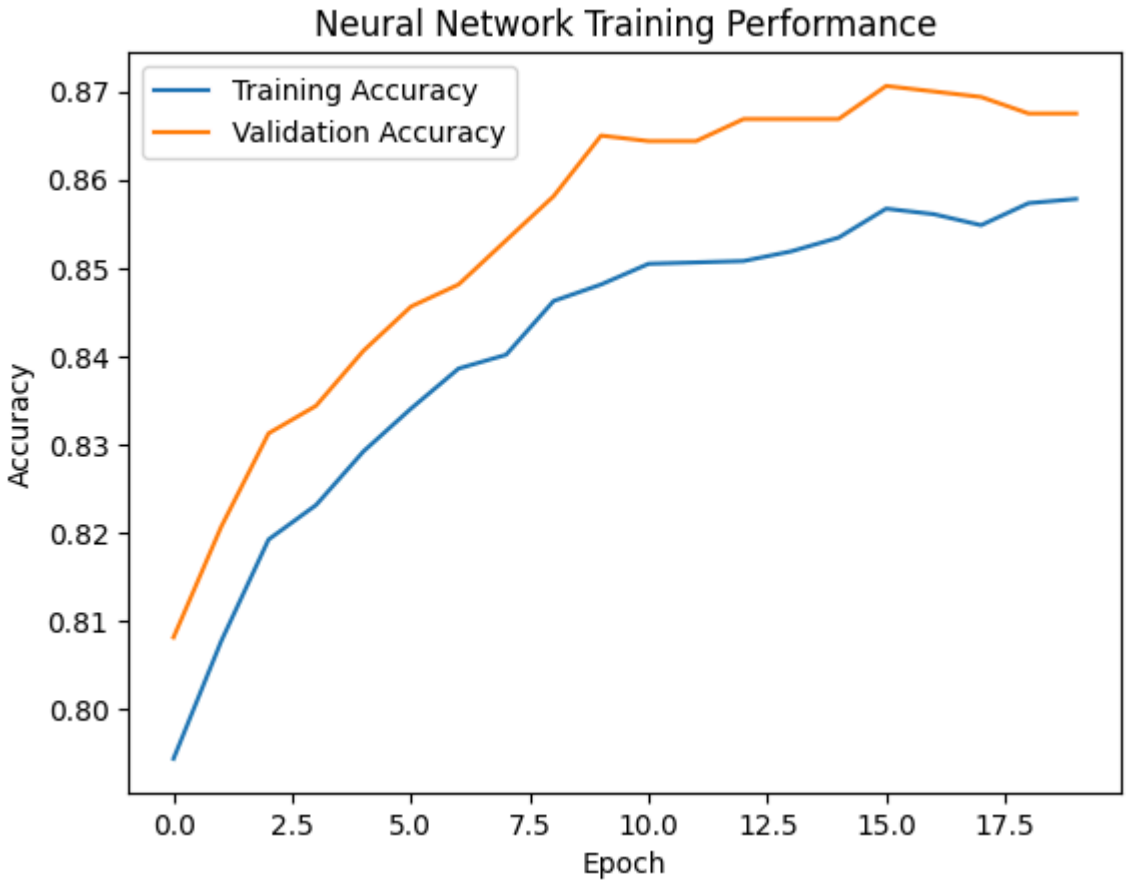
```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Accuracy: 0.8575

Confusion Matrix:
[[1539   54]
 [ 231  176]]
```

```
import matplotlib.pyplot as plt

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Neural Network Training Performance")
plt.legend()
plt.show()
```



Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 4

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    precision_score, recall_score
)
```

```
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

```
Choose Files diabetes.csv
diabetes.csv(text/csv) - 23105 bytes, last modified: 21/10/2026 - 100% done
Saving diabetes.csv to diabetes.csv
Dataset Shape: (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
print("\nClass Distribution:")
print(y.value_counts())
```

```
Features Shape: (768, 8)
Target Shape: (768,)

Class Distribution:
Outcome
0    500
```

```
1    268
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (614, 8)
Testing Data: (154, 8)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Data normalization completed.")
```

```
Data normalization completed.
```

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("KNN model trained successfully.")
```

```
KNN model trained successfully.
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("Accuracy:", round(accuracy, 4))
print("Error Rate:", round(1 - accuracy, 4))

print("Precision:",
      round(precision_score(y_test, y_pred, zero_division=0), 4))

print("Recall:",
      round(recall_score(y_test, y_pred, zero_division=0), 4))
```

```
Confusion Matrix:
[[80 20]
 [26 28]]
Accuracy: 0.7013
Error Rate: 0.2987
Precision: 0.5833
Recall: 0.5185
```